LOGICFLOW: THE DIGITAL ARCHEOLOGIST

Complete Technical Architecture, System Features, Self-Healing Pipeline & Performance Analysis

Project: Major Project Final	**Target LLM:** Ollama (llama3.2:3b)
Architecture: Autonomous RAG + AST Surgical Agent	**Execution Time:** < 60s (Sub-1-Minute Engine)
Supported Languages: Python, C, C++, Java, JS, TS, Go	**Verification:** Local AST, GCC, G++, Javac & Docker

1. Executive Summary & Problem Statement

LogicFlow (subtitled *'The Digital Archeologist'*) is a self-contained, autonomous code repair and self-healing engineering agent. It is designed to navigate complex codebases, discover logic and compilation bugs, and deploy mathematically validated patches without human intervention. Unlike generic AI code generators that rewrite entire source files, LogicFlow implements a **surgical AST (Abstract Syntax Tree) isolation architecture**. It focuses lightweight local models (such as llama3.2:3b) strictly on broken routines while leaving surrounding production logic completely intact.

Critical Problems Solved by LogicFlow:

- **Hallucination & Code Truncation:** Conventional LLMs truncate 100+ line files with comments like *'// rest of code unchanged'*. LogicFlow's AST splicer guarantees zero file truncation.
- **Multi-Minute CI/CD Delays:** High token limits and Docker pull network checks previously caused 5-minute latencies. LogicFlow's token budgeting and direct local compiler paths achieve complete repairs in **under 42 seconds**.
- **Cross-Language Translation Errors:** Lightweight models frequently convert C or Java into Python. LogicFlow enforces strict multi-language preservation prompts and localized compiler diagnostics.
- **Logic Bug Repeating (Anti-Regression):** Small models often output the exact same erroneous code on retries. LogicFlow incorporates a self-healing feedback loop with previous failed code reflection and dynamic temperature escalation.

2. System Architecture & Core Subsystems

LogicFlow is architected as a modular, decoupled pipeline comprising eight dedicated subsystems:

Subsystem	Source File	Core Responsibilities & Capabilities
Orchestration Agent	`agents/core.py`	Unified state machine (AgentState, AgentResult). Orchestrates repair loops, token budgeting, prompt construction, security auditing, and PR publishing.

AST Surgical Engine	`tools/ast_tool.py`	Parses source into Python ASTs, locates failing functions/classes, parses stack traces, and splices candidate patches back into source code with exact indentation.
Multi-Compiler Sandbox	`tools/docker_test_tool.py`	Two-tier verification engine. Fast local compiler routing (GCC, G++, Javac, Node, Python AST) with containerized Docker sandboxing fallback. Automatic test discovery.
RAG & Semantic Search	`tools/faiss_tool.py`	FAISS vector database with sentence-transformers (all-MiniLM-L6-v2). Language-aware structural context trimming (macros, structs, function signatures).
GitHub Integration	`tools/github_tool.py`	Fetches remote issues, parses stack traces, extracts referenced files, creates isolated git branches, and submits pull requests via PyGithub or local Git CLI fallback.
Configuration Engine	`config.py`	Centralized configuration with environment variable support. Defines LLM endpoints, compiler commands, Docker images, resource limits, and token budgets.
Interactive Dashboard	`dashboard/streamlit_app.py`	Full web UI supporting manual code uploads, syntax pre-checks, autonomous test suites, GitHub issue triaging, side-by-side diff viewers, and log streaming.

3. Operational Modes & Key Features

Mode 1: Autonomous Test & Repair

Executes the entire workspace test suite without human prompts. When a test fails, the stack trace is parsed to extract the exact file and broken function. The agent isolates the function, synthesizes a patch, validates it in the sandbox, and automatically commits the verified fix or opens a GitHub Pull Request.

Mode 2: Local File Targeted Repair

Direct file-level repair requiring only a target filename and an error log or assertion failure. Enables rapid developer fixing during local development. Automatically searches workspace subfolders and links corresponding unit tests for regression verification.

Mode 3: GitHub Issue Workflow

Connects to remote GitHub repositories, fetches open bug reports, identifies culprit files via semantic RAG or regex file matching, runs unit tests upfront to extract assertion failures, repairs the code, and automatically opens a comprehensive Pull Request with fix explanations and security notes.

Mode 4: Upload & Real-Time Auto-Fix

Interactive web workbench in Streamlit. Users paste or upload single files (.py, .c, .cpp, .java, .js, .ts). Features automatic language detection, real-time syntax pre-checking via local compilers (GCC/Javac/Python), and generates side-by-side visual diffs with download options.

4. Technical Deep-Dive: How LogicFlow Works Under the Hood

4.1 AST Surgical Isolation & Function Splicing

The core innovation in LogicFlow is its ability to perform **micro-surgical repairs**. When an error occurs in a 500-line file, sending the entire file to a lightweight 3B model guarantees failure: the model runs out of context tokens, truncates code, or introduces subtle regressions. LogicFlow executes the following AST pipeline:

1. **Traceback Parsing:** ASTExtractor.parse_traceback() parses Python stack traces, regex patterns, or unit test assertion outputs to extract (target_file, line_number, function_name).
2. **AST Node Extraction:** ast.parse() traverses the syntax tree, identifies the target FunctionDef or AsyncFunctionDef, and extracts exact source coordinates.
3. **Outline Generation:** ASTExtractor.get_file_outline() extracts class and function signatures across the rest of the file so the LLM understands surrounding context without consuming token budget.
4. **AST Splicing:** Once the LLM returns the corrected function, ASTExtractor.replace_function_in_source() calculates the exact line offsets, matches the original indentation, replaces the broken function block, and validates syntactic integrity with ast.parse().

4.2 Multi-Language Auto-Detection & Class Name Alignment

To support diverse software stacks without human configuration, LogicFlow incorporates token-level language heuristics and file naming compliance rules:

- **Token Heuristics:** Distinguishes C (#include <stdio.h>, printf), C++ (std::, cout), Java (public class, System.out), Python (def, import), and JavaScript/TypeScript.
- **Java Class-Filename Sync:** The Java compiler enforces that public class Foo must reside in Foo.java. LogicFlow parses top-level class names via regex and automatically names the compilation target to prevent compiler mismatches.
- **Strict Language Preservation Prompts:** Dynamic system prompts explicitly command the model: *'The target is {LANG}. Never translate or convert to Python or any other language!'*

4.3 Upfront Test Diagnostics & The Self-Healing Feedback Loop

A common failure mode in AI agents is repeating the same broken code on retries. LogicFlow prevents this through an **anti-regression self-healing loop**:

1. **Upfront Test Execution:** Before invoking the LLM for a GitHub issue or local repair, LogicFlow executes the associated test suite (e.g. test_token_bucket_rate_limiter.py) on the existing code to capture exact assertion messages and line numbers.
2. **Candidate Validation:** The LLM's candidate fix is tested in the sandbox. If verification passes, it moves to security audit. If it fails, the error log is captured.
3. **Previous Failed Code Injection:** On Attempt 2, the feedback prompt explicitly provides the model with the *exact failed code it wrote in Attempt 1* alongside the failure trace, with the instruction: *'Do NOT repeat the previous implementation! Modify the broken condition.'*
4. **Dynamic Temperature Escalation:** Temperature escalates from 0.1 (greedy/deterministic) on Attempt 1 to 0.35 on Attempt 2, forcing the model to explore alternate algorithmic logic.

4.4 Token Budgeting & Sub-1-Minute Optimization

On local consumer hardware (CPU or integrated GPUs), inference throughput for 3B parameter models is approximately 12–18 tokens per second. Capping max_tokens to **768 for surgical mode** and **1024 for whole files** reduces LLM generation time from 140s to ~15–25s. Furthermore, routing compilation checks to local binaries (GCC, Javac, Python) bypasses Docker network pull stalls, achieving instant **0.05-second verification**.

5. Performance Benchmarks & Empirical Results

LogicFlow was benchmarked against real-world test scenarios including concurrency bugs, algorithm flaws, and cross-language repairs:

Scenario / Benchmark	Language	Bug Type	Attempts	Execution Time	Outcome
Token Bucket Rate Limiter	Python	Logic Bug (Balance corrupted on rejected burst requests)	1	**41.44 sec**	**Passed & Verified**
Student Management System	C	Compiler Error (Implicit declaration of print vs printf)	2 (Self-heal)	**38.12 sec**	**GCC Syntax Verified**
Bank Account & Calculator	Java	Compilation Error (Method typo & public class alignment)	1	**29.80 sec**	**Javac 21 Verified**
E-Commerce Discount Engine	Python	Assertion Failure (Negative discount boundary error)	1	**34.50 sec**	**Test Suite Green**

6. Security Architecture & Guardrails

Autonomous code modification agents present inherent security risks if unconstrained. LogicFlow integrates a defense-in-depth safety architecture:

- **System File Guardrails:** _execute_repair_loop() enforces a strict blocklist rejecting attempts to modify system libraries or interpreter internals (site-packages, unittest, pytest, importlib, python3, AppData).
- **Docker Container Isolation:** When containerized execution is active, containers run with --network=none, a 512MB memory limit, and CPU quota limits (cpu_period=100000, cpu_quota=50000) to prevent infinite loops, fork bombs, or credential exfiltration.
- **OWASP Security Audit:** Every candidate fix undergoes an automated vulnerability evaluation (_evaluate_security()) checking for SQL/command injection, buffer overflows, path traversal, hardcoded secrets, and unsafe deserialization.

7. Technology Stack & Project Structure

Category	Technologies Used	Role in System
LLM Inference	Ollama, Llama 3.2 (3B parameters)	Local, privacy-preserving, zero-cloud-cost reasoning engine.
AST & Parsing	Python ast, traceback, regex	Surgical function extraction, outline generation, and AST splicing.
Vector Search	FAISS, sentence-transformers	Semantic code indexing and contextual dependency retrieval.
Compilers	MinGW GCC, G++, Javac 21, Python AST	Instant syntax and compilation verification (< 0.1s latency).
Sandboxing	Docker (isolated containers, network disabled)	Quarantined code execution and multi-environment testing.
Version Control	PyGithub, Git CLI, GitHub REST API	Issue triaging, automated branching, and Pull Request creation.
User Interface	Streamlit, HTML5, Custom CSS	Interactive operational workbench and real-time execution monitor.

8. Conclusion & Key Takeaways

LogicFlow demonstrates that autonomous, reliable software engineering does not require massive 70B+ cloud models. By pairing a lightweight 3B local model with **rigorous compiler verification, Abstract Syntax Tree surgical splicing, and anti-regression feedback loops**, LogicFlow achieves rapid, verified code repairs in **under 45 seconds**. It provides a resilient foundation for the next generation of autonomous CI/CD pipelines, self-healing repositories, and developer productivity tools.